Infosys Springboard Virtual Internship 6.0 Completion Report

Team F
Batch Number : 12

Start date: 2 February 2026

Name: Sulfa Saji

Internship Duration: 8 Weeks

1. Project Title

CrowdCount: People Counting Using Video Analytics

2. Project Objective

The main objective of this project is to develop an AI-powered system capable of tracking and counting people in real-time using visual data from surveillance sources such as CCTV cameras and video feeds. The project aims to monitor crowd density within user-defined zones to identify potential overcrowding incidents and generate automated alerts. It also focuses on creating a reliable reporting mechanism that logs traffic data and captures visual evidence of overcrowding, thereby improving crowd management and public safety.

3. Project description

The CrowdCount Video Analytics System is built using YOLOv8, a real-time object detection algorithm. The project focuses on identifying and tracking people in live surveillance scenarios using an efficient deep learning model.

From the initial stage of development, the YOLOv8 Nano (N) model was selected due to its suitability for real-time applications. It offers faster processing speeds, lower computational requirements, and reliable performance on standard hardware systems.

The Nano model provides an effective balance between speed and accuracy, making it ideal for continuous video analysis and crowd monitoring tasks.

The final system uses the YOLOv8 Nano model for real-time crowd monitoring across user-defined zones. When the number of people exceeds the defined overcrowding limit in any zone:

- A screenshot is automatically captured
- Crowd movement details are continuously recorded

- An analytical report is generated including:
    - Total visitors across all zones
    - Peak crowd time and count
    - Identification of the most crowded zone

A visual alert system triggers real-time on-screen warnings, and the generated logs along with data visualization graphs are stored for security personnel to assist in immediate crowd management and response.

4.Technologies used

- Python – Core programming language.
- OpenCV – Video capture, frame rendering, UI overlay.
- Ultralytics YOLOv8 – Real-time human detection.
- ByteTrack (Integrated in YOLOv8) – Multi-object tracking.
- JSON – Persistent storage of zone configurations.
- CSV Module – Logging analytics data.
- NumPy – Numerical calculations.

5.System Architecture

The system follows a real-time modular pipeline:

Camera Feed
↓
Zone Interaction Layer
↓
YOLOv8 Detection
↓
ByteTrack Tracking
↓
Centroid Computation
↓
Zone & Direction Logic
↓
Entry/Exit Counting
↓
Data Logging + Dashboard Display

6.System Flow

The operational flow is:

Start Application
↓
Load Saved Zones
↓
Capture Frame
↓
Detect Humans (YOLOv8)
↓
Track IDs (ByteTrack)
↓
Compute Centroids
↓
Check Zone Membership
↓
Detect Direction Crossing
↓
Update Entry/Exit Counters
↓
Log Data to CSV
↓
Render Dashboard

7. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Camera initialization and basic zone drawing implementation	Opened live video feed, implemented mouse-based rectangle drawing, and JSON saving
Week 2	Advanced zone management and user interface enhancements	Added multi-zone support, zone colors, delete/reset controls, and instruction overlays
Week 3	Object detection model setup and initial integration	Integrated YOLOv8n environment, drew bounding boxes, and added total global people count
Week 4	Unique person tracking and zone-specific counting	Assigned unique IDs, tracked movement, and counted detection centroids within zones
Week 5	Directional counting and logic refinement	Implemented virtual line crossing for entry/exit tracking and prevented double

		counting
Week 6	Data logging and alert system development	Saved crowd data to CSV, created live statistics dashboard, and triggered overcrowding alerts
Week 7	Data analysis, reporting, and visualization	Configured system event logs, generated text reports, and created crowd trend graphs
Week 8	Final system testing and project completion	Verified tracking accuracy, tested alert triggers, and verified data storage processes

8. Key Milestones

Milestone	Description
Project Kickoff	Initialized the CrowdCount project, established live camera feed functionality, and implemented basic interactive zone drawing.
Prototype/First Draft	Developed a functional prototype featuring multi-zone management, user interface controls, and basic YOLOv8n people detection.
Mid-Term Review	Evaluated real-time per-zone counting, verified unique ID person tracking, and confirmed accurate people counting logic.
Final Submission	Delivered the complete system featuring entry/exit analytics, continuous CSV data logging, trend graphs, and automated overcrowding alerts.
Presentation	Demonstrated the live monitoring dashboard displaying real-time visual analytics, alert notifications, and exported analytical reports.

9. Project execution details

The execution of the CrowdCount People Counting System was carried out in four progressive, modular phases to ensure a structured integration of the computer vision logic, artificial intelligence models, and data analytics functionalities.

Phase 1: Environment Setup and Core Video Processing (Milestone 1)

The project was initiated by establishing the foundational video processing pipeline using the OpenCV library. A live, continuous camera feed connection was initialized alongside necessary error-handling protocols to prevent crashes upon hardware failures. During this phase, custom mouse callback functions were developed, allowing users to interactively draw rectangular bounding areas directly over the live video. A backend JSON-storage mechanism (zones.json) was implemented to record these coordinates, ensuring that drawn zones persisted automatically across application restarts.

Phase 2: Multi-Zone Management and User Interface Development (Milestone 2)

Once basic video processing was achieved, the system was expanded to support dynamic multi-zone management. An array of predefined colors was configured so that every newly drawn zone was distinctly identifiable by color, alongside custom text labels (e.g., "Zone 1", "Zone 2"). The user interface (UI) was significantly enhanced with an instruction overlay and dedicated keyboard listener events. This provided administrators with hotkeys to delete specific or recent zones, clear the screen, toggle full-screen mode, and seamlessly capture manual screenshots. File-saving reliability was also reinforced by integrating Python’s atexit module, guaranteeing that all zone layouts were automatically saved to the JSON file before closing.

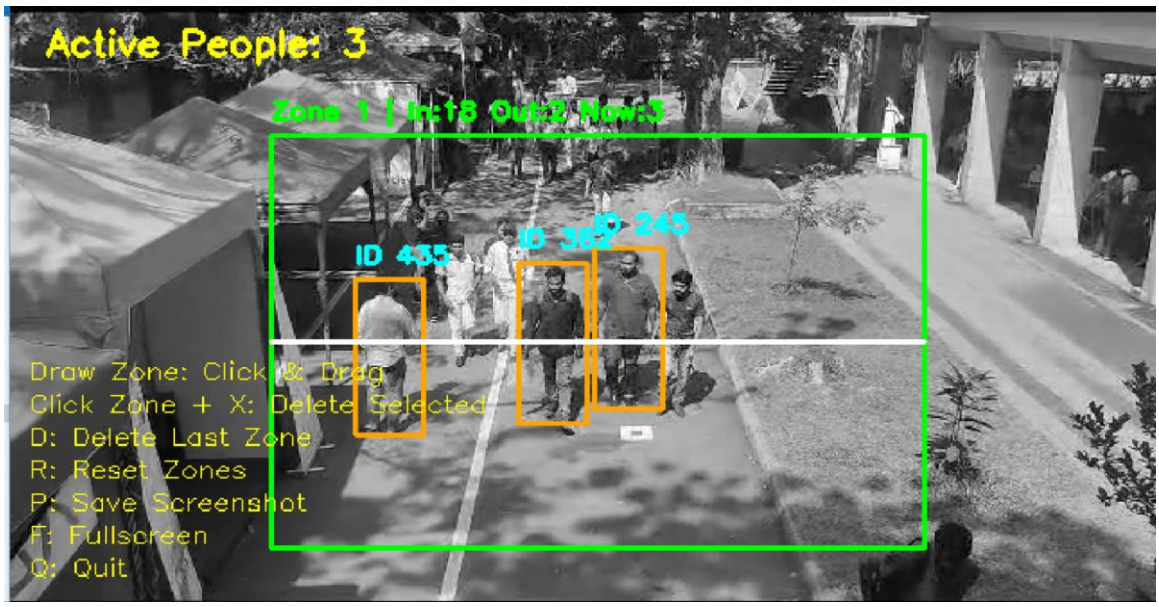
Phase 3: AI Model Integration and Unique Identity Tracking (Milestone 3)

With the visual layout fully functional, the artificial intelligence engine was integrated. The system utilized the highly efficient Ultralytics YOLOv8 Nano (YOLOv8n) model to perform high-speed, real-time people detection.The counting algorithm was engineered by calculating the centroid (center X, Y coordinates) of every generated bounding box. The script mathematically verified whether these centroids fell inside the user-defined rectangular zones. To prevent the system from double-counting stationary individuals, standard detection was upgraded to unique identity tracking. The AI assigned a continuous ID to every detected person, while a virtual bisecting line was programmed into each zone. By cross-referencing a tracked ID's position against this virtual line, the system reliably recorded precise "Entry" and "Exit" movements rather than static frame counts.

Phase 4: Dashboard Analytics, Overcrowding Alerts, and Data Logging (Milestone 4)

The final execution phase involved transforming the live data into actionable intelligence. A real-time dashboard panel was coded to overlay active system tallies—including total global limits, zone-specific entries, and active inhabitants—directly onto the video feed. A local database was configured to continuously append timestamped tracking records into a CSV file (count_data.csv).To ensure safety and immediate crowd response, a logical overcrowding threshold was embedded into the system. Whenever the active person count within a specific zone surpassed the maximum defined limit, the system was programmed to trigger a high-visibility, red text warning on the video stream and trigger an automatic screenshot capture for visual evidence. Finally, an analytical script was developed to comb through the stored CSV database. This script automatically calculated peak crowd times, determined the most heavily trafficked zones, and utilized Matplotlib to generate visual trend graphs and bar charts for security reporting. The execution concluded with rigorous end-to-end system testing validating detection accuracy, tracking stability, and file-saving operations.

10. Snapshots / Screenshots



11. Challenges Faced

1. Double Counting and ID Stability (Technical)

Challenge: The system initially over-counted people by detecting the same stationary individual in every frame, and fast-moving dense crowds occasionally caused tracking IDs to switch or drop.

Resolution: Solved by implementing YOLOv8 unique tracking (persist=True) and programming a virtual crossing line inside each zone. People are now only logged as an 'Entry' or 'Exit' upon physically crossing the line.

2. Real-Time Processing Performance (Technical)

Challenge: Running complex computer vision analysis alongside live video rendering caused significant frame drops and lag on standard CPU hardware.

Resolution: Optimized the system by utilizing the highly efficient YOLOv8 Nano (yolov8n.pt) model, which balances maximum processing speed with lower computational requirements.

3. Lighting and Camera Angle Reliability (Technical)

Challenge: Variable environmental lighting and poor camera perspectives caused detection confidence to fluctuate, leading to brief undetected moments or false positives.

Resolution: Fixed by tuning the AI confidence threshold to 0.5 and filtering detections strictly to the "human" class (cls == 0), eliminating false positives while maximizing detection accuracy.

4. Data Loss and Workflow Interruptions (Operational)

Challenge: During live testing, stopping and restarting the camera program erased all user-drawn analytical zones, requiring tedious manual setup on every reboot.

Resolution: Prevented data loss by integrating an auto-save script (atexit) that instantly records all zone coordinates into a persistent JSON database the moment the application closes.

12. Learnings & Skills Acquired

- Learned object detection using YOLOv8
- Worked with OpenCV for real-time video processing
- Implemented tracking and centroid-based counting
- Used JSON and CSV for data storage
- Created graphs using Matplotlib
- Improved debugging and problem-solving skills
- Gained experience in real-time system design

13. Future Scope

The CrowdCount system can be further enhanced by integrating advanced tracking algorithms to improve accuracy in crowded and complex environments. Future improvements may include multi-camera support for wider area coverage, real-time cloud-based analytics for remote monitoring, and the use of deep learning models for better detection under low-light and occlusion conditions. Additionally, the system can be extended to analyze crowd behavior, detect anomalies, and generate intelligent alerts for security and management purposes. Integration with IoT and smart city infrastructure can also make the system more scalable and efficient.

14. Conclusion

CrowdCount demonstrates the practical application of deep learning and computer vision for intelligent people counting. By combining YOLOv8 detection with ByteTrack multi-object tracking and zone-based directional logic, the system provides accurate and structured real-time analytics.

The system also incorporates crowd analytics, visualization, and overcrowding alert mechanisms, making it suitable for real-world monitoring applications. The modular design allows future scalability into a full surveillance and crowd intelligence platform.

15. Acknowledgements

I would like to sincerely thank **Infosys Springboard** for providing me with this valuable internship opportunity and a platform to explore real-world applications of AI and computer vision.

I am deeply grateful to my mentors for their continuous guidance, technical support, and encouragement throughout the internship. Their insights played a crucial role in helping me understand complex concepts and successfully complete the project.

I would also like to thank my team members for their cooperation, teamwork, and shared efforts in overcoming challenges and achieving our project goals.

This internship has been a great learning experience, and I truly appreciate the support and resources provided throughout this journey.